

Intro-to-ICSlab

李晓慧 信息南楼A412
lxh2023@ouc.edu.cn

October 10, 2026

Press Space for next page →

时间安排

实验	周次	截至时间	资料
环境配置	2	虚拟机安装、GNU工具链使用	PPT
lab0	2	第4周截止	PPT
lab1:datalab发布	4	第6周截止	实验手册
lab2:bomblab发布	6	第10周截止	实验手册
lab2:bomblab检查 **lab3:attacklab发布**	8	第12周截止	实验手册

时间安排

实验	周次	截至时间	资料
lab3:attacklab检查	10	第12周截止	实验手册
lab3:attacklab检查 **lab4:汇编实验发布**	12	第14周截止	实验手册
lab4:汇编实验检查	14	第14周截止	实验手册
实验小测	16		

实验简介

- 成绩说明：实验总共占20分。
- 其中，lab0 2’，datalab 5’, bomblab 7’，attacklab 3’, 汇编实验 2’
- 实验考试2’（含附加分1’）

实验报告要求

- 无实验报告模板
- 命名方式：`labX-学号-姓名.pdf` 如 `lab2-25020011010-张三.pdf`
- 内容上要有**完整截图、实验总结、实验思考**，缺一不可
- 提交平台：雨课堂

环境配置

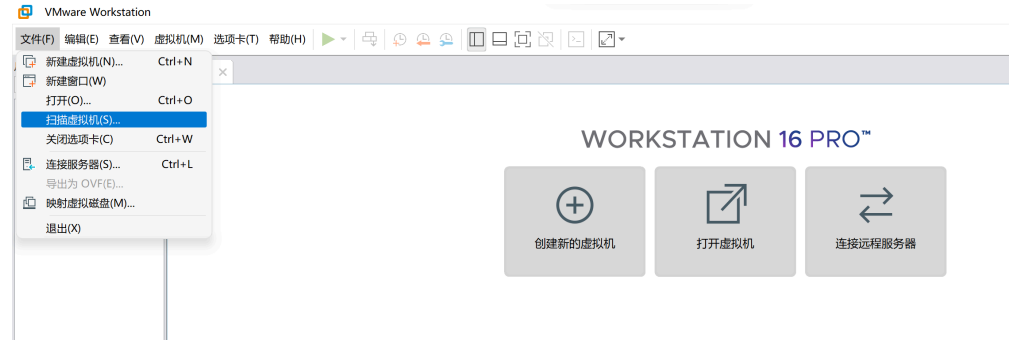
1. 虚拟机资源

- 本课程提供打包好的虚拟机下载地址（请使用校园网访问）
- 虚拟机解压所在盘需保证至少有20G空间
- 虚拟机已做好基本的配置，开盒即用

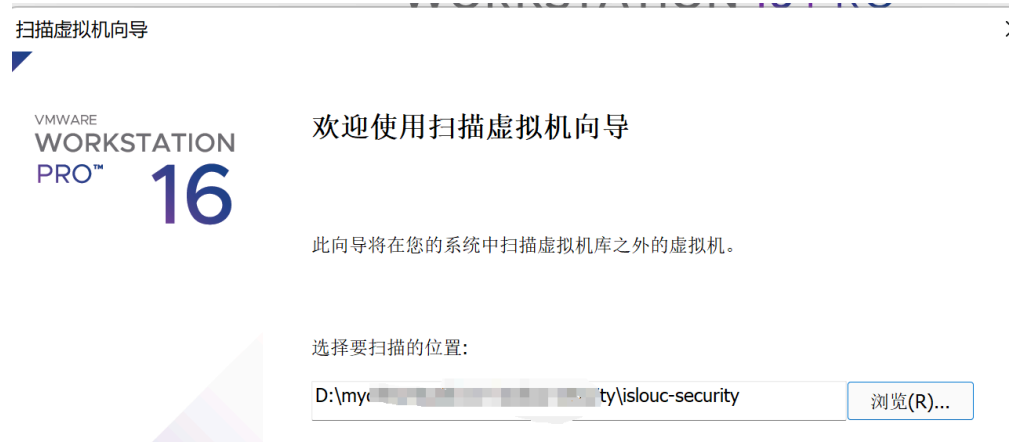
2. vmware workstation

官网下载vmware workstation，并安装 自行上网找破解码

- 启动 `vmwareworkstation` –“文件”–“扫描虚拟机”



- 扫描位置选择上述虚拟机解压后的所在目录



- 在vm中成功打开虚拟机，第一次打开会显示“运行此虚拟机”，以后打开都显示“继续运行此虚拟机”，虚拟机**描述**中有账号密码。虚拟机第一次启动有一定耗时。
  - linux常用命令.pdf
  - vim使用手册.pdf

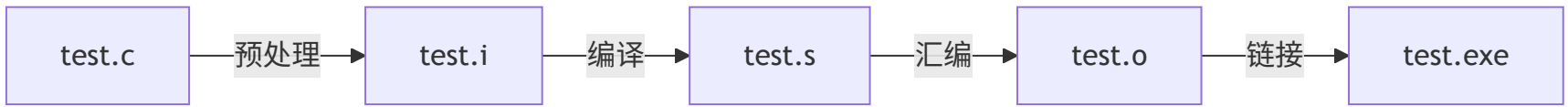
---

3. 本地与虚拟机之间的文件共享

- 在本地复制`file1`，进入虚拟机界面，右键–“paste”，不出意外的话可以直接粘贴；
- 若无法直接复制粘贴，可在虚拟机配置中**设置共享文件夹**；
- 或使用scp命令 `scp [文件名(也可以是绝对路径)] [虚拟机用户名，不一定是root]@[虚拟机ip]:[虚拟机的文件路径]`
  - 如 `scp c:\file1 ouc@192.168.xx.xx:/home/`，
  - 虚拟机地址可通过“右键–terminal–输入`ifconfig`命令”确定
- 或自行查找其他方法

预备知识

二进制基础–从源文件到可执行程序



1. GNU binutils:

- GCC、objdump

2. GNU debugger:

- GDB

GCC

GCC是Linux上的常用编译工具，支持多种编程语言。

```
$ gcc -v
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=/usr/lib/gcc/x86_64-linux-gnu
OFFLOAD_TARGET_NAMES=nvptx-none:hsa
OFFLOAD_TARGET_DEFAULT=1
Target: x86_64-linux-gnu
Configured with: ../src/configure -v --with-pkgve
```

- `gcc main.c -o main` 编译单个文件
- `gcc main.c -o main && ./main` 编译并运行
- 分阶段
  - `gcc -E main.c -o main.i` 预处理
  - `gcc -S main.i -o main.s` 生成汇编文件
    - `cc1 main.i -o main.s`
  - `gcc -c main.s -o main.o` 生成目标文件
    - `as main.s -o main.o`
  - `gcc main.o -o main` 链接生成可执行文件
    - `ld main.o -o main -lc -dynamic-linker /lib64/ld-linux-x86-64.so.2 ...`
- 是否允许调试（借助GDB）
  - `gcc -g main.c -o main`

反汇编工具-objdump

常见的汇编代码分为 AT&T 风格（目的操作数在右）和 Intel 风格（目的操作数在左）两种

- 给定一个二进制文件，我们可以使用反编译工具得到他们的汇编代码，如 `objdump` 。
  - 默认情况下 `objdump -d myhelloworld` 生成AT&T风格的汇编
  - `objdump` 提供 `--intel` 选项，支持生成Intel风格的汇编

OBJDUMP的一般使用

- `objdump -d <binary-file>`
  - `-d` :表示 `--disassemble` 反汇编 `.text` 段
  - `-D` :表示 `--disassemble-all` 反汇编所有代码段

```
# 使用objdump的--disassemble=sym 指定符号
attacklab$ objdump --disassemble=main ctarget

ctarget:     file format elf64-x86-64


Disassembly of section .init:

Disassembly of section .plt:

Disassembly of section .plt.sec:

Disassembly of section .text:

0000000000401779 <main>:
  401779:       f3 0f 1e fa             endbr64
  40177d:       41 56                   push   %r
  40177f:       41 55                   push   %r
  401781:       41 54                   push   %r
```

GDB调试

> 调试二进制程序，支持多种编程语言

程序可调试的条件

- `gcc -g` 允许在目标文件中加入调试信息
  - 包括变量、函数的数据类型
  - 源代码行号与可执行代码地址的关联
- `strip`
  - 对编译好的程序，剥离符号信息
  - 调试会变得很困难

```
$ gdb myhello

# 设置参数，需要在程序运行之前
(gdb) set args 参数1 参数2 ...
(gdb) show args # 查看设置的命令行参数

# run==r
(gdb)run # 运行，直到第一个断点位置
(gdb)start #在main函数前设置断点，并运行
```

- 查看代码

```
# list==l
(gdb)list # 查看文件代码，从第一行开始显示
(gdb)list 行号 # 从指定行号显示一定的上下文代码，默认是显示10
(gdb)list myfunction
```

- 设置断点

```
# break==b
(gdb)break myfunction #根据函数名设置断点
(gdb)break 5 # 行号

# delete==d 删除断点
(gdb)d 1 # 删除第一个断点
(gdb)d 1-5 # 删除第1到第5个断点
```

- 调试

```
# continue==c
(gdb)c # 从当前位置继续执行，直到下一个断点
# step==s
(gdb)s # 单步调试，进入函数体内部
# next==n
(gdb)n # 单步调试，但不进入内部
(gdb)finish # 跳出函数内部
(gdb)until # 跳出循环体
```

打印变量

```
# print==p 打印变量的值
(gdb)p 变量名
# 整数，可指定输出格式
# /x 十六进制、/d 十进制、/u 无符号十进制
# /t 二进制、/f 浮点数、/c 字符
(gdb)p/fmt 变量名 # fmt表示整数格式，如p/x

(gdb)ptype 变量名 # 打印变量类型
```

查看信息

```
# info==i 查看状态/信息
# registers==r
(gdb)info breakpoint/break/b #查看断点信息
(gdb)info registers # 查看寄存器信息
(gdb)info frame # 查看栈帧
(gdb)info args # 查看参数
(gdb)info locals # 查看局部变量

# disas 反汇编
(gdb)disas myfunctionname

# 单元长度u:
# b 一个字节、h 两个字节（半字）
```

Welcome to ICS-lab0

lab0

> 熟悉命令行环境、编译工具、反汇编工具
> 给定源程序，进行编译与反编译分析

- 源程序 (文本) `hello.c`
- 源程序 (文本) `hello.i`
- 汇编语言程序 (文本) `hello.s`
- 可重定位目标程序 (二进制) `hello.o`
- 可执行目标程序 (二进制) `hello`
- 反汇编程序1 `objdump -S hello.o>hello.txt`
- 反汇编程序2 `objdump -S hello>helloo.txt`
- 程序输出结果对比分析

源代码

```c
#include "stdio.h"

int main(){
  int x=3, y=5, z;
  z = x+y;
  printf("z=%d\n", z);
  return 0;
}
```

实验步骤1

```
$ mkdir ics
$ cd ics
$ mkdir lab0
$ cd lab0
# 进入 ics/lab0 文件夹

$ vim hello.c
# 创建hello.c源文件，复制/输入代码

$ gcc -O0 -m32 -g hello.c -o a
# -O0 不进行编译优化，-m32 IA-32指令，
# -g 带调试信息，搭配后续的GDB调试
# 或者增加 -no -pie -fno -pic 选项

$ objdump -S a>a.txt
# 反汇编，-S 保留C语句，> 保存到文件
# 用编辑器打开 a.txt
# 根据 C 语言读懂汇编代码，认识指令
```

实验步骤2

```
$ gdb a
# 启动 GDB 调试工具，进入 a 的 GDB 调试环境

(gdb) break main
# 在 main 函数入口处设置断点
(gdb) break a.c:3
# 源程序 a.c 的第 3 行处设置断点

(gdb) run  # 启动运行程序

(gdb) si  # (或s) 执行一条指令（语句）

# 查看所有定点寄存器内容
(gdb) i r
# 或者，查看给定的寄存器内容
(gdb) i r esp ebp # 或者 (gdb) i r eax eflags

# 重复执行以下步骤，观察栈中数据变化
(gdb) x/yxb $esp     # 按字节显示当前栈帧内
# y=(ebp)-(esp)+4     # 当前栈帧的字节单元数
(gdb) x/zxw $esp     # 按4字节显示当前栈帧内容
# z=((ebp)-(esp)+4)/4     # 当前栈帧的4字节单元数

# 退出调试
(gdb)quit
```